

## PROBLEM 1: THE LAST ASTRONAUT

PROGRAM The\_Last\_Astronaut

DECLARE

N

K

astronauts

count

currentIndex

currentK

eliminated

i

BEGIN

GET N

GET K

FOR i FROM 0 UNTIL N - 1

SET astronauts[i] = i + 1

END FOR

SET count = N

SET currentIndex = 0

SET currentK = K

PRINT "Urutan astronot yang dieliminasi:"

WHILE count > 1

SET currentIndex = (currentIndex + currentK - 1) MOD count

SET eliminated = astronauts[currentIndex]

PRINT eliminated

```

FOR i FROM currentIndex UNTIL count - 2
    SET astronauts[i] = astronauts[i + 1]
END FOR

SET count = count - 1

IF eliminated MOD 2 == 0 THEN
    SET currentK = currentK + 2
ELSE
    SET currentK = currentK - 1
END IF

IF currentK < 2 THEN
    SET currentK = 2
END IF

END WHILE

PRINT "Astronot terakhir yang bertahan:", astronauts[0]

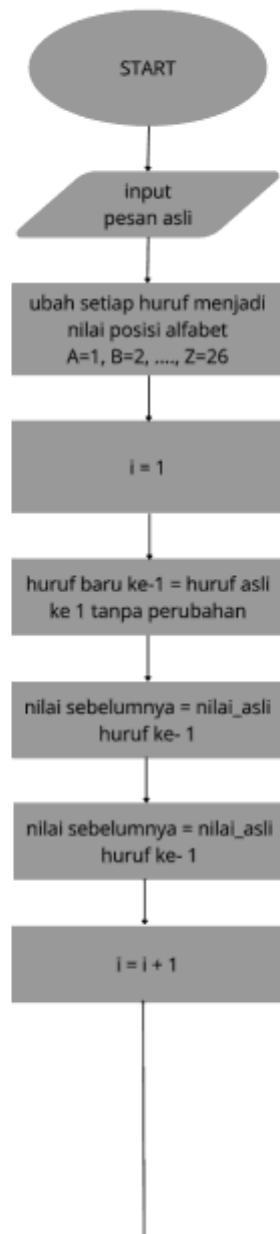
END

```

Algoritma *The Last Astronaut* diawali dengan menerima masukan  $N$  sebagai jumlah astronot dan  $K$  sebagai nilai hitungan awal, lalu mengisi *array* *astronauts* dari angka 1 hingga  $N$  untuk merepresentasikan posisi unik setiap astronot di dalam lingkaran. Tiga variabel penelusur utama yang disiapkan mencakup *count* untuk memantau jumlah astronot yang aktif, *currentIndex* sebagai penanda posisi hitungan awal, dan *currentK* untuk menyimpan nilai langkah hitungan terkini. Simulasi ini dijalankan melalui perulangan *WHILE* yang berputar selama *count* masih lebih besar dari 1. Pada setiap putaran, indeks astronot yang akan dieliminasi dihitung menggunakan fungsi modulo  $(currentIndex + currentK - 1) \bmod count$  agar hitungan melingkar tetap berada dalam jangkauan *array* aktif.

Setelah indeks tereliminasi ditentukan, nomor astronot pada *astronauts[currentIndex]* dicetak sebagai urutan eliminasi, lalu seluruh elemen *array* di sebelah kanannya digeser satu langkah ke kiri untuk merapatkan barisan. Variabel *count* kemudian dikurangi 1 untuk memperbarui sisa populasi astronot. Selanjutnya, algoritma memeriksa sifat ganjil-genap dari nomor astronot yang tersingkir tersebut: jika nomornya genap, nilai *currentK* ditambah

2, dan jika ganjil, `currentK` dikurangi 1. Sebuah batas kondisi diterapkan untuk memastikan jika hasil pengurangan menyebabkan `currentK` kurang dari 2, maka nilainya otomatis diubah menjadi 2. Seluruh alur eliminasi dan penyesuaian nilai  $K$  ini berulang hingga `count` menyisakan 1 astronot, dan elemen tersisa pada `astronauts[0]` dicetak sebagai astronot terakhir yang bertahan.





## PROBLEM 2: ALIEN IN THE MIDLE

### 1. Konversi huruf ke angka

Setiap huruf pada pesan diubah ke nilai posisinya di alfabet:

A = 1, B = 2, C = 3, ..., Z = 26.

## 2. Huruf pertama tidak berubah

Huruf pertama dari pesan langsung dijadikan huruf baru tanpa ada perhitungan apa pun. Ini karena tidak ada "huruf sebelumnya" yang bisa dijadikan acuan pergeseran.

Contoh: pesan ALIENS, huruf pertama A  $\rightarrow$  tetap A.

## 3. Huruf-huruf berikutnya digeser berdasarkan huruf sebelumnya

Untuk huruf ke-2 dan seterusnya, rumusnya:

Hasil = nilai\_posisi(huruf sekarang) + nilai\_posisi ASLI(huruf sebelumnya)

Poin pentingnya: "huruf sebelumnya" yang dipakai adalah nilai aslinya dari pesan asli, bukan nilai barunya yang sudah disandikan. Ini yang sering bikin bingung/salah kalau tidak diperhatikan.

Contoh dari tabel:

- Huruf ke-2: L (=12). Huruf sebelumnya (asli) adalah A (=1). Hasil =  $12 + 1 = 13 \rightarrow$  M.
- Huruf ke-3: I (=9). Huruf sebelumnya (asli) adalah L (=12), *bukan* M yang sudah jadi huruf baru. Hasil =  $9 + 12 = 21 \rightarrow$  U.
- Begitu seterusnya, huruf sebelumnya yang dipakai selalu nilai asli dari pesan asli, mengikuti urutan aslinya.

## 4. Penanganan jika hasil melebihi Z (26)

Karena alfabet hanya sampai Z = 26, jika hasil penjumlahan melebihi 26, maka dikurangi 26 supaya kembali berputar dari A.

Contoh: huruf ke-6, S (=19), huruf sebelumnya asli N (=14).

Hasil =  $19 + 14 = 33 \rightarrow$  karena  $> 26$ , maka  $33 - 26 = 7 \rightarrow$  huruf ke-7 = G.

## 5. Ulangi sampai semua huruf selesai

Proses ini diulang dari huruf ke-2 sampai huruf terakhir pesan, dengan setiap langkah:

1. Ambil nilai asli huruf sekarang.
2. Tambahkan dengan nilai asli huruf tepat sebelumnya (dari pesan asli, bukan hasil sandi).
3. Jika hasil  $> 26$ , kurangi 26.
4. Ubah hasil menjadi huruf sesuai posisi alfabetnya — itulah huruf sandi untuk posisi tersebut.

Ringkasan alur berpikir program

- Simpan pesan asli dalam bentuk array huruf/angka.
- Buat array/variabel untuk menyimpan hasil sandi.
- Loop dari indeks ke-2 hingga akhir, karena indeks pertama sudah pasti sama.
- Di setiap iterasi, gunakan nilai *asli* pesan pada indeks sebelumnya (pesan[i-1]), jangan memakai hasil sandi yang baru dihitung.
- Terapkan pengecekan modulo/pengurangan 26 agar tetap valid A–Z.

Kalau Anda mau, saya bisa lanjutkan buat pseudocode atau langsung kode program (C/Python/dll sesuai bahasa tugas TLS) berdasarkan logika ini.